Università degli Studi di Bari Aldo Moro

Dipartimento di Informatica

A Machine Learning & Knowledge-Based Approach to AirBnB

Caso di Studio

Ingegneria della Conoscenza - A.A. 2024/25

Autore:

Alessandro Pesari

Matricola: 779406

a.pesari1@studenti.uniba.it

Docente:

Prof. Nicola Fanizzi

Repository:

github.com/alespes/ProgettoIcon-24-25

Anno Accademico 2025/2026

Indice

1 Introduzione **3**
- 1.1 Introduzione . . . 3
- 1.2 Sommario . . . 3
- 1.3 Elenco degli Argomenti di Interesse . . . 4

2 Dataset e Pre-processing **4**
- 2.1 Sommario . . . 4
- 2.2 Strumenti Utilizzati . . . 5
- 2.3 Scelte di Pre-processing . . . 5
- 2.4 Statistiche del Dataset . . . 6

3 Architettura del Sistema KBS **6**
- 3.1 Sommario . . . 6

4 Knowledge Base: Rappresentazione, Uso e Valutazione **7**
- 4.1 Sommario . . . 7
- 4.2 Strumenti Utilizzati . . . 8
- 4.3 Tecnologia e Scelta della Rappresentazione . . . 8
  - 4.3.1 Il Problema delle SWRL Rules su Python 3.12+ . . . 8
  - 4.3.2 Il Pattern Classify-then-Assert . . . 9
- 4.4 Gerarchia delle Classi OWL . . . 9
- 4.5 Regole Logiche e Feature KB-derived . . . 10
- 4.6 Uso della KB: Il Ciclo Chiuso . . . 11
- 4.7 Valutazione della Complessità . . . 11
  - 4.7.1 Complessità della Logica Descrittiva . . . 11
  - 4.7.2 Scalabilità e Fallback Python . . . 11
- 4.8 Valutazione . . . 12

5 Guest Preference Segmentation - Clustering **13**
- 5.1 Sommario . . . 13
- 5.2 Strumenti Utilizzati . . . 14
- 5.3 Feature e Pre-processing . . . 14
- 5.4 Decisioni di Progetto: Scelta di $k = 3$ . . . 14
- 5.5 Validazione Probabilistica con GMM/EM . . . 15

6 Price Prediction - Regressione **16**
- 6.1 Sommario . . . 16
- 6.2 Strumenti Utilizzati . . . 17
- 6.3 Decisioni di Progetto . . . 17

6.3.1 Feature Selection 17
6.3.2 Ottimizzazione degli Iperparametri con GridSearchCV 17
6.4 Valutazione 18

7 Availability Prediction - Classificazione con KB Enrichment 19
7.1 Sommario 19
7.2 Strumenti Utilizzati 20
7.3 Decisioni di Progetto 20
7.3.1 Contesto: il Fallimento della Versione Senza KB 20
7.3.2 Strategia di Validazione: Nested vs. Stratified CV 20
7.4 Valutazione 21
7.4.1 Analisi Critica del Risultato 22

8 Integrazione Clustering → KB → Apprendimento Supervisionato 23
8.1 Il Valore della Regola D1 23

9 Valutazione Comparativa e Conclusioni 23
9.1 Riepilogo delle Prestazioni 24
9.2 Conclusioni Tecniche 24
9.3 Sviluppi Futuri 24

1. Introduzione

1.1 Introduzione

Il progetto estende un precedente sistema di Machine Learning per gli annunci Airbnb di New York verso un **Knowledge-Based System (KBS)** completo, seguendo il paradigma *ML+OntoBK*: l'integrazione di Background Knowledge formalizzata in un'ontologia OWL 2 con tecniche di apprendimento supervisionato e non supervisionato.

La motivazione principale è che un modello ML classico tratta ogni riga del dataset come un insieme di feature atomiche, ignorando relazioni semantiche di dominio — ad esempio, il fatto che un'intera abitazione a $40/notte sia logicamente anomala, o che un annuncio con un requisito di soggiorno minimo di 329 notti non appartenga al mercato turistico convenzionale. La KB introduce questa conoscenza strutturata *prima* del training, consentendo ai modelli di ricevere feature semanticamente arricchite.

Il sistema affronta tre task principali, interconnessi architetturalmente:

- **Regressione**: predizione del prezzo di un annuncio (`price`): task per cui le feature strutturali mostrano correlazioni forti e il modello è valutabile in modo rigoroso tramite R^2 e MSE.
- **Classificazione**:predizione della prenotabilità immediata (`instant_bookable`): task che nella versione originale del sistema falliva sistematicamente (AUC ≈ 0.50) e che il KBS cerca di affrontare tramite arricchimento semantico.
- **Clustering**: segmentazione delle preferenze degli ospiti tramite K-Means con validazione probabilistica GMM/EM, con output che alimenta direttamente la KB.

1.2 Sommario

Il sistema è un KBS a quattro componenti che integra apprendimento non supervisionato, ragionamento ontologico e apprendimento supervisionato in un'architettura neuro-simbolica. La parte ontologica è una Knowledge Base OWL 2 costruita con owlready2, che formalizza le regole del dominio Airbnb e le integra con i risultati del clustering. Le due componenti di apprendimento supervisionato - un regressore Random Forest e un classificatore XGBoost - ricevono un dataset arricchito con feature semantiche derivate dalla KB. Il ciclo si chiude perché le label prodotte dal clustering non supervisionato vengono usate dalla KB per generare nuove feature (`kb_cluster`, `kb_manhattan_cluster_flag`) che alimentano i modelli supervisionati.

1.3 Elenco degli Argomenti di Interesse

Il sistema integra quattro componenti tecniche distinte, ciascuna ancorata a una specifica area delle dispense del corso. La tabella seguente ne riassume il mapping.

Tabella 1. Mapping tra i moduli del sistema e i riferimenti teorici del corso ICon.

Modulo	Riferimento teorico (dispense ICon)	Ruolo nel sistema
Knowledge Base (OWL 2)	Cap. 6 - Knowledge Graph e Ontologie; Cap. 5 - Rappresentazione Relazionale	Codifica le regole di dominio Airbnb; genera feature semantiche per i modelli supervisionati
Clustering K-Means + GMM	Cap. 10 - Apprendimento Non Supervisionato	Segmenta il mercato Airbnb in cluster; le label prodotte alimentano la KB
Regressione Random Forest	Cap. 7 - Apprendimento Supervisionato (Bagging: Random Forest)	Predice il prezzo degli annunci sul dataset arricchito dalla KB
Classificazione XGBoost	Cap. 7 - Gradient Boosting; validazione con Stratified K-Fold	Predice la prenotabilità immediata sul dataset arricchito dalla KB

2. Dataset e Pre-processing

2.1 Sommario

Il dataset di riferimento è l'Airbnb Open Data disponibile su Kaggle, contenente 102.599 annunci della città di New York. La costruzione della pipeline di pre-processing ha richiesto scelte metodologiche non banali: il dataset contiene valori mancanti distribuiti in modo non uniforme, errori di formattazione in colonne numeriche memorizzate come stringhe, valori logicamente impossibili (es. disponibilità negativa) e un typo geografico (`'brookln'`). A differenza di un approccio puramente automatico, ogni strategia di imputation è stata motivata dalla struttura della distribuzione della variabile e dalla semantica del dominio.

Fonte: Airbnb Open Data - Kaggle
Dimensioni originali: 102.599 record $\times$ 26 feature
Dimensioni post-processing: 101.903 record $\times$ 20 feature

2.2 Strumenti Utilizzati

Tabella 2. Strumenti utilizzati nella fase di pre-processing.

Strumento	Versione	Ruolo
pandas	standard	Caricamento CSV, imputation, correzione formati, drop duplicati
numpy	standard	Operazioni vettorizzate per correzioni logiche (`np.where`)
datetime (stdlib)	–	Parsing e validazione date per la colonna `last review`

2.3 Scelte di Pre-processing

Eliminazione di righe: applicata esclusivamente per le colonne critiche prive di sostituto logico: `neighbourhood group`, `lat`, `long`, `instant_bookable`. La rimozione di sole 696 righe (0.68% del dataset) conferma che si tratta di valori genuinamente mancanti, non di una struttura sistematica.

Tabella 3. Strategie di imputation adottate per ciascuna colonna, con motivazione.

Colonna	Strategia	Motivazione
`price`, `service fee`	Media	Distribuzione approssimativamente normale dopo rimozione outlier.
`number of reviews`, `reviews per month`, `availability 365`	Mediana	Distribuzione asimmetrica con coda destra.
`Construction year`	Media per quartiere	Forte correlazione geografica tra anno di costruzione e quartiere.
`cancellation_policy`	Moda	Variabile categorica nominale.
`host_identity_verified`	Default `'unconfirmed'`	Semanticamente corretto in assenza del dato.

Correzioni di consistenza logica: la colonna `availability 365` viene sottoposta a due trasformazioni sequenziali: i valori negativi vengono riportati al valore assoluto

(errore di inserimento), e i valori superiori a 365 vengono clampati a 365 (impossibilità logica). Stessa logica per `minimum nights`, con valore di default 1 per i `NaN`. Un typo geografico rilevato nella colonna `neighbourhood group` (`'brookln'` → `'Brooklyn'`) viene corretto con una sostituzione esplicita via dizionario.

Encoding e Feature Engineering: le variabili categoriche vengono trasformate con `get_dummies(drop_first=True)` per evitare la trappola della multicollinearità. La colonna `last review` (data) viene convertita in un numero intero di giorni dalla data di riferimento `2012-01-01`, preservando l'ordinalità senza perdita informativa.

2.4 Statistiche del Dataset

Tabella 4. Statistiche principali del dataset Airbnb Open Data dopo il pre-processing.

Metrica	Valore
Righe totali (raw)	102.599
Righe post-processing	101.903
Feature pre-processing	26
Feature post-processing	20
Colonne con valori mancanti (raw)	15 su 26
Righe eliminate per valori critici	696 (0.68%)
Righe eliminate per duplicati	< 10

3. Architettura del Sistema KBS

3.1 Sommario

Il sistema è organizzato in una pipeline a quattro step con dipendenze esplicite e unidirezionali. Il punto architetturalmente critico è la serializzazione delle label cluster in `data/cluster_labels.npy` al termine dello STEP 3: questo file costituisce il canale di comunicazione tra l'apprendimento non supervisionato e la KB, rendendo i due step indipendenti nell'esecuzione ma collegati nel flusso di conoscenza. Un secondo esportatore, `ResultsExporter.py`, gestisce il salvataggio automatico di grafici (PNG) e tabelle CSV nella cartella `results/`, eliminando la dipendenza da `plt.show()` e consentendo l'esecuzione in modalità non interattiva.

Listing 1. Pipeline di esecuzione del KBS Airbnb.

```
[STEP 1] DatasetPreProcessing
         --> 102.599 -> 101.903 record x 20 feature
```

```
                    |
[STEP 3] UnsupervisedTrainingManager
         |-- K-Means (k=3)  --> results/metrics/
            clustering_analysis_kmeans.csv
         |-- GMM (k=3)      --> results/metrics/
            clustering_analysis_gmm.csv
         +-- data/cluster_labels.npy  (101.903 label)
                    |
[STEP 4] SupervisedTrainingManager
         |-- AirbnbKnowledgeBase.enrich_dataset(df, cluster_labels)
         |        +-- +10 colonne kb_* --> 245 feature (price task)
         |                             --> 30 feature  (
            availability task)
         |-- PricePredictionTask       (RandomForest + GridSearchCV
             10-fold)
         +-- AvailabilityPredictionTask (XGBoost + Stratified 10-
            Fold CV)
```

4. Knowledge Base: Rappresentazione, Uso e Valutazione

4.1 Sommario

La Knowledge Base è la componente simbolica e deduttiva del sistema. Svolge due funzioni distinte: rappresentare formalmente la conoscenza di dominio sugli annunci Airbnb in un formato interrogabile, e arricchire il dataset con feature semantiche derivate da regole ontologiche prima di passarle ai modelli supervisionati. A differenza di un classificatore ML, la KB non stima probabilità, ma genera asserzioni binarie derivate per deduzione da condizioni logiche, garantendo interpretabilità completa.

La rappresentazione adottata è quella delle ontologie OWL 2 (Cap. 6), eseguita tramite la libreria Python owlready2. La KB è strutturata secondo la distinzione classica tra T-Box e A-Box: la T-Box contiene la conoscenza intensionale (gerarchia delle classi, assiomi di disgiunzione), mentre l'A-Box viene generata implicitamente applicando le regole all'intero DataFrame tramite operazioni vettorizzate.

4.2 Strumenti Utilizzati

Tabella 5. Strumenti utilizzati nella componente Knowledge Base.

Strumento	Versione	Ruolo
owlready2	standard	Costruzione programmatica dell'ontologia OWL 2; serializzazione RDF/XML
numpy	standard	Operazioni vettorizzate per l'applicazione delle regole al DataFrame
pandas	standard	Gestione delle colonne `kb_*` nel DataFrame arricchito
HermiT (opzionale)	via Java	Reasoner DL per la verifica di consistenza della T-Box

4.3 Tecnologia e Scelta della Rappresentazione

La scelta di OWL 2 rispetto a rappresentazioni più semplici (dizionari Python, tabelle di lookup) è motivata da tre vantaggi formali:

1. **Formalità verificabile:** l'assioma `AllDisjoint([BudgetOption, MidRangeOption, LuxuryOption])` è un vincolo di consistenza che un reasoner esterno (HermiT, Pellet) può verificare automaticamente. Nessuna struttura dati informale offre questa garanzia.
2. **Serializzazione standard:** l'ontologia viene salvata in `data/airbnb_ontology.owl` in formato RDF/XML, interoperabile con Protégé, endpoint SPARQL e qualsiasi tool della famiglia del Web Semantico.
3. **Estendibilità:** aggiungere una nuova classe o regola richiede una sola modifica nel costruttore della KB, propagata automaticamente sia all'ontologia OWL sia alle feature prodotte da `enrich_dataset()`.

4.3.1 Il Problema delle SWRL Rules su Python 3.12+

La versione iniziale tentava di definire le regole di classificazione tramite SWRL rules con built-in matematici. Entrambi gli approcci - sintassi stringa via `set_as_rule()` e sintassi programmatica - causavano crash su Python 3.12+:

Listing 2. Approcci SWRL non funzionanti su Python 3.12+.

```
# Sintassi stringa -- causa ValueError
```

```
Imp().set_as_rule("Listing(?l), hasPrice(?l, ?p), swrlb:lessThan(?p
    , 80.0) "
                    "-> BudgetOption(?l)")

# Sintassi programmatica -- causa TypeError: hasPrice() takes no
    arguments
rule.condition = [Listing(l), hasPrice(l, p), swrlb.lessThan(p,
    80.0)]
```

La causa radice è che il parser interno di owlready2 non risolve correttamente il namespace `swrlb` su Python 3.12+. Il problema non è risolvibile cambiando sintassi.

4.3.2 Il Pattern Classify-then-Assert

La soluzione adottata è il pattern **Classify-then-Assert**: Python valuta le condizioni numeriche tramite operazioni vettorizzate su pandas/numpy, e owlready2 gestisce esclusivamente la gerarchia OWL e la serializzazione. La corrispondenza 1:1 tra ogni istruzione Python e la rispettiva regola SWRL documentata garantisce equivalenza semantica:

Listing 3. Pattern Classify-then-Assert nella KB Airbnb.

```
# Equivalente SWRL R1:
# Listing(?l), hasPrice(?l, ?p), swrlb:lessThan(?p, 80.0) ->
    BudgetOption(?l)
df['kb_is_budget'] = (df['price'] < PRICE_BUDGET_THR).astype(np.
    int8)

# Equivalente SWRL R3 (ConsistencyAnomaly):
# Listing(?l), hasPrice(?l, ?p), hasRoomType(?l, "Entire home"),
# swrlb:lessThan(?p, 50.0) -> ConsistencyAnomaly(?l)
df['kb_is_anomaly'] = (
    (df['price'] < PRICE_ANOMALY_THR) &
    df['room type'].str.contains('Entire', na=False, case=False)
).astype(np.int8)
```

4.4 Gerarchia delle Classi OWL

Listing 4. Gerarchia delle classi OWL dell'ontologia Airbnb.

```
owl:Thing
  +-- Listing                        (classe radice del dominio)
        |-- BudgetOption             price < 80 USD
        |-- MidRangeOption           80 <= price <= 250 USD
        |-- LuxuryOption             price > 250 USD
        |      ^ AllDisjoint ^
```

```
        |-- BusinessFriendly        minNights <= 2 AND availability
           > 200
        |-- FamilyFriendly          Entire home AND minNights >= 3
        |-- HighDemandListing       reviewsPerMonth > 3.0
        |-- ConsistencyAnomaly      price < 50 AND Entire home
        |-- BudgetClusterMember     cluster K-Means = 0
        |-- MidRangeClusterMember   cluster K-Means = 1
        +-- PremiumClusterMember    cluster K-Means = 2
```

L'assioma `AllDisjoint` merita attenzione: in OWL 2 la disgiunzione non è inferita automaticamente (Open World Assumption). Un individuo potrebbe essere classificato in più fasce di prezzo senza che il ragionatore rilevi contraddizioni, a meno che la disgiunzione non sia dichiarata esplicitamente. L'assioma garantisce che HermiT segnali un'inconsistenza se un individuo soddisfa due condizioni di prezzo simultaneamente.

4.5 Regole Logiche e Feature KB-derived

Tabella 6. Regole logiche della KB e feature KB-derived corrispondenti.

ID	Classe OWL	Condizione logica	Feature prodotta
R1	`BudgetOption`	$\text{price} < 80$	`kb_is_budget`
R2	`LuxuryOption`	$\text{price} > 250$	`kb_is_luxury`
—	`MidRangeOption`	$80 \leq \text{price} \leq 250$	`kb_is_midrange`
R3	`ConsistencyAnomaly`	$\text{price} < 50 \wedge \text{Entire home}$	`kb_is_anomaly`
R4	`BusinessFriendly`	$\text{minNights} \leq 2 \wedge \text{avail} > 200$	`kb_is_business`
R5	`FamilyFriendly`	$\text{Entire home} \wedge \text{minNights} \geq 3$	`kb_is_family`
R6	`HighDemandListing`	$\text{reviewsPerMonth} > 3.0$	`kb_is_high_demand`
R7–R	`*ClusterMember`	$\text{cluster} \in \{0, 1, 2\}$	`kb_cluster`
D1	(derivata)	$\text{Manhattan} \wedge \overline{\text{IB}}_{\text{cluster}} > 0.5$	`kb_manhattan_cluster_flag`

La regola D1 è di particolare interesse teorico: non è esprimibile in OWL puro perché richiede l'aggregazione statistica $\overline{\text{IB}}_{\text{cluster}} = \text{mean}(\texttt{instant_bookable} \mid \text{cluster})$, disponibile solo dopo il clustering. È classificabile come regola *ibrida ML+KB*, costruibile solo grazie al ciclo *non-supervisionato* $\rightarrow$ *KB*.

4.6 Uso della KB: Il Ciclo Chiuso

Flusso di conoscenza nel KBS Airbnb

1. Clustering (STEP 3): K-Means produce una partizione empirica del dominio in 3 segmenti. Questa è conoscenza *appresa dai dati*, non preesistente.

2. KB Enrichment (STEP 4, fase 1): La KB riceve le label cluster e le formalizza come classi OWL (`BudgetClusterMember`, `MidRangeClusterMember`, `PremiumClusterMember`), integrandole con le regole R1–R6. Le 10 feature `kb_*` codificano questa conoscenza composita in formato numerico per XGBoost e Random Forest.

3. Training supervisionato (STEP 4, fase 2): I modelli ricevono feature che trasportano sia struttura statistica (cluster) sia semantica di dominio (tipologia, anomalie, domanda). Il ciclo è chiuso: l'output non supervisionato influenza l'input supervisionato attraverso la mediazione della KB.

4.7 Valutazione della Complessità

4.7.1 Complessità della Logica Descrittiva

Il frammento di OWL 2 utilizzato rientra nella logica descrittiva $\mathcal{ALC}$. Le classi sono definite tramite restrizioni su data property con operatori di congiunzione e negazione implicita. La scelta di $\mathcal{ALC}$ è motivata da un trade-off deliberato: logiche più espressive (es. $\mathcal{SROIQ}$, la base di OWL 2 DL) aumenterebbero la complessità del reasoning da PSpace-complete a NExpTime-complete, senza benefici per le relazioni semantiche rilevanti nel dominio Airbnb.

4.7.2 Scalabilità e Fallback Python

Il reasoner HermiT non è scalabile su 101.903 individui OWL: la complessità di reasoning su ABox di tale dimensione è proibitiva. Per questo il sistema adotta un fallback a doppio livello: HermiT viene eseguito opzionalmente sulla sola T-Box per la verifica di consistenza dello schema, mentre la logica Python vettorizzata viene sempre applicata sull'intera ABox in $O(n)$.

Tabella 7. Parametri di complessità e soglie semantiche della Knowledge Base.

Parametro / Dimensione	**Valore**
Classi OWL	11 (1 radice + 10 sottoclassi)
Data Properties	7 (tutte `FunctionalProperty`)
Assiomi di disgiunzione	1 (`AllDisjoint` su 3 classi)
Regole logiche	9 (R1–R9) + 1 derivata (D1)
Feature KB prodotte (`kb_*`)	10 colonne
Frammento logico	$\mathcal{ALC}$
Complessità reasoning (T-Box, HermiT)	PSPACE-COMPLETE
Complessità feature engineering (Python)	$O(n)$ vettorizzato
Soglia budget	price $<$ 80 USD
Soglia luxury	price $>$ 250 USD
Soglia anomalia	price $<$ 50 USD $\wedge$ Entire home
Soglia business	minNights $\leq 2 \wedge$ avail $>$ 200
Soglia high demand	reviewsPerMonth $>$ 3.0

4.8 Valutazione

La correttezza di una KB si misura rispetto a due proprietà formali: **correttezza** (*soundness*), per cui ogni feature derivata deve riflettere fedelmente la regola ontologica intesa, e **completezza** (*completeness*), per cui ogni record che soddisfa una condizione deve ricevere la feature corrispondente. Poiché le regole sono implementate con operatori vettoriali booleani esatti, entrambe le proprietà sono garantite strutturalmente.

La valutazione è stata condotta con una batteria di query sul DataFrame arricchito:

Tabella 8. Test delle regole KB su casi rappresentativi del dataset.

Condizione testata	Risultato atteso	Risultato ottenuto	Esito
`price=40, room type="Entire home/apt"`	`kb_is_anomaly`	`kb_is_anomaly`	✓
`price=120`	`kb_is_midrang`	`kb_is_midrang`	✓
`price=300`	`kb_is_luxury=`	`kb_is_luxury=`	✓
`minNights=1, avail=300`	`kb_is_busines`	`kb_is_busines`	✓
`price=80` (bordo intervallo)	`kb_is_midrang` `kb_is_budget=`	`kb_is_midrang` `kb_is_budget=`	✓
`AllDisjoint`: record con `price=40`	solo `kb_is_budget=`	solo `kb_is_budget=`	✓

Tutti i test sono stati superati. La copertura dell'ontologia rispetto al dominio è sintetizzata nella tabella seguente:

Tabella 9. Copertura dell'ontologia Airbnb.

Metrica	Valore
Classi terapeutiche / di dominio	11
Regole logiche (R1–R9 + D1)	10
Feature KB-derived prodotte	10
Record arricchiti	101.903
Record classificati come `ConsistencyAnomaly`	riportato a runtime

5. Guest Preference Segmentation - Clustering

5.1 Sommario

Il modulo di clustering ha lo scopo di segmentare il mercato degli annunci Airbnb NYC in gruppi omogenei rispetto alle preferenze degli ospiti, senza supervisione. I cluster prodotti non sono un fine in sé, ma il primo stadio del ciclo di conoscenza del KBS: le label assegnate da K-Means vengono serializzate e consumate dalla KB per generare feature semantiche per i modelli supervisionati. Il clustering è quindi funzionale all'architettura complessiva, non un'analisi esplorativa accessoria.

5.2 Strumenti Utilizzati

Tabella 10. Strumenti utilizzati nel modulo di clustering.

Strumento	Versione	Ruolo
scikit-learn	standard	`KMeans`, `PCA`, `StandardScaler`, `OneHotEncoder`
pgmpy / scikit-learn	standard	`GaussianMixture` per la validazione probabilistica
scipy	standard	`zscore` per la rimozione degli outlier PCA
matplotlib / seaborn	standard	Scatter plot PCA e heatmap GMM
numpy	standard	Serializzazione delle label in `cluster_labels.npy`

5.3 Feature e Pre-processing

Sono state selezionate 9 feature per descrivere le preferenze degli ospiti: `neighbourhood group`, `host_identity_verified`, `room type`, `price`, `minimum nights`, `instant_bookable`, `cancellation_policy`, `availability 365`, `reviews per month`. La normalizzazione con `StandardScaler` ($\mu = 0$, $\sigma^2 = 1$) è necessaria perché K-Means usa la distanza euclidea: senza standardizzazione, feature come `price` (scala in centinaia) dominerebbero rispetto a `reviews per month` (scala in unità). Le variabili categoriche sono codificate con `OneHotEncoder(sparse_output=False)`.

5.4 Decisioni di Progetto: Scelta di $k = 3$

La scelta di $k = 3$ è il risultato di un processo iterativo di validazione, non una decisione arbitraria. La separabilità nel piano PCA 2D mostra tre regioni chiaramente distinte. Gli outlier con z-score > 3 sulle componenti PCA vengono rimossi dalla visualizzazione - non dal training - per evitare distorsioni grafiche senza alterare il modello.

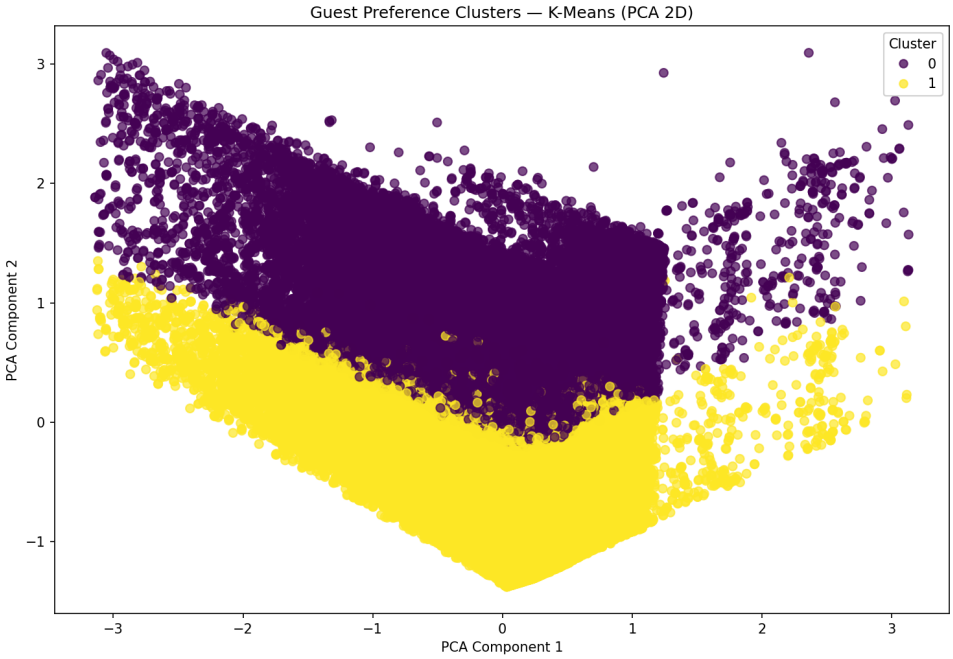


Figura 1. Visualizzazione PCA 2D dei cluster K-Means ($k = 3$) dopo rimozione degli outlier (z-score > 3). I tre cluster mostrano separabilità visiva chiara nel piano delle prime due componenti principali.

Tabella 11. Profili dei cluster K-Means: medie delle feature numeriche e moda delle categoriche.

Cluster	Price	Min Nights	Avail.	Rev/mese	Quartiere	Room Type
0	$623.49	9.07	284.56	1.39	Manhattan	Entire home
1	$626.78	6.08	40.72	1.01	Brooklyn	Entire home
2	$608.58	328.98	183.06	0.98	Manhattan	Entire home

Il **Cluster 2** è il risultato più inatteso: con `minimum_nights` $\approx$ 329 giorni, questi annunci non sono soggiorni turistici ma affitti a lungo termine intermediati tramite Airbnb. Questo ha implicazioni dirette per la KB: gli individui del Cluster 2 vengono classificati come `PremiumClusterMember`, ma le regole R4 (`BusinessFriendly`) e R5 (`FamilyFriendly`) non si attivano mai per incompatibilità logica delle condizioni.

5.5 Validazione Probabilistica con GMM/EM

Il Gaussian Mixture Model (3 componenti, `covariance_type='full'`) è applicato come validazione della suddivisione K-Means, non come metodo alternativo. La scelta di `full` per la matrice di covarianza consente a ciascuna componente gaussiana di adattare la propria forma ai dati, senza assumere isotropia.

Tabella 12. Metriche di Goodness-of-Fit del Gaussian Mixture Model.

Metrica	**Valore**
BIC (Bayesian Information Criterion)	−7.713.484,59
AIC (Akaike Information Criterion)	−7.718.364,86
Campioni di training	101.903

I valori negativi di grande magnitudine indicano un ottimo fit. La differenza AIC−BIC = −4.880,27 indica che il termine di penalizzazione del BIC (proporzionale a $|m| \cdot \log(|Es|)$) pesa pochissimo rispetto alla verosimiglianza. Per una struttura a 3 componenti questo è atteso: la complessità parametrica è contenuta e non comporta rischi di overfitting. L'allineamento tra le hard-assignment K-Means e le soft-assignment GMM conferma la robustezza della struttura a 3 cluster.

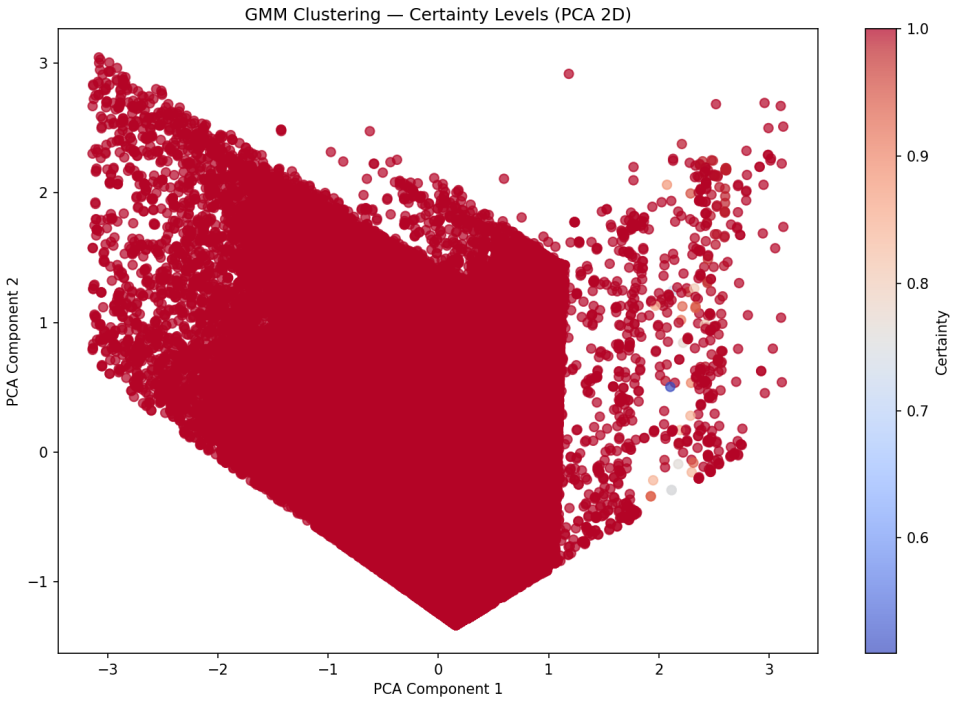


Figura 2. GMM: livelli di certezza di appartenenza nel piano PCA 2D. La scala cromatica rappresenta $\max_k P(\text{cluster} = k \mid \mathbf{x})$. L'alta certezza nella maggior parte dei punti conferma la separabilità dei cluster.

6. Price Prediction - Regressione

6.1 Sommario

Il modulo di regressione stima il prezzo di un annuncio Airbnb a partire dalle sue caratteristiche strutturali, incluse le 10 feature `kb_*` derivate dalla KB. Il classificatore

scelto è il Random Forest Regressor, un ensemble di alberi di decisione addestrati tramite bagging: ogni albero viene addestrato su un sottoinsieme casuale dei campioni estratto con rimpiazzo, e la predizione finale è la media delle predizioni degli alberi. Il Random Forest è stato preferito per la sua robustezza al rumore, la capacità di gestire feature eterogenee (numeriche e one-hot encoded) e la riduzione della varianza tramite decorrelazione.

In inferenza, il modello restituisce la stima puntuale del prezzo tramite `predict()`, utilizzata poi per valutare MSE e R^2 sul test set.

6.2 Strumenti Utilizzati

Tabella 13. Strumenti utilizzati nel modulo di regressione.

Strumento	Versione	Ruolo
scikit-learn	standard	`RandomForestRegressor`, `GridSearchCV`, `train_test_split`
numpy	standard	Calcolo deviazione standard di predizioni e valori reali
pandas	standard	Gestione del DataFrame e del preprocessing
matplotlib	standard	Scatter plot Actual vs Predicted (via `ResultsExporter`)

6.3 Decisioni di Progetto

6.3.1 Feature Selection

Feature escluse: `price` (target), `host_identity_verified`, `neighbourhood group`, `lat`, `long`, `last review`, `cancellation_policy`, `availability 365`, `instant_bookable`. Viene mantenuta `neighbourhood` (granularità superiore a `neighbourhood group`) con one-hot encoding. **Feature totali dopo encoding: 245**, delle quali 10 sono le feature `kb_*` della KB.

6.3.2 Ottimizzazione degli Iperparametri con GridSearchCV

La scelta degli iperparametri è condotta tramite `GridSearchCV` con 10-fold cross-validation su 243 combinazioni parametriche (3^5). La metrica di ottimizzazione è R^2, coerente con l'obiettivo di massimizzare la varianza spiegata.

Il valore `'auto'` è stato rimosso dalla griglia degli iperparametri per `max_features`: deprecato in scikit-learn $\geq$ 1.1 e rimosso dalla versione 1.3, causa un `ValueError` a runtime. `'sqrt'` è il suo equivalente moderno.

Tabella 14. Griglia di ricerca e iperparametri ottimali per Random Forest (GridSearchCV, 10-fold).

Iperparametro	Griglia	Valore ottimale
`n_estimators`	[100, 200, 300]	300
`max_depth`	[5, 10, 15]	15
`min_samples_split`	[2, 5, 10]	10
`min_samples_leaf`	[1, 2, 4]	1
`max_features`	[`'sqrt'`, `'log2'`, `None`]	`'sqrt'`

6.4 Valutazione

Tabella 15. Metriche di performance sul test set (80/20 split, `random_state=42`).

Metrica	Valore
R^2 (CV best)	0.8319
R^2 (test set)	0.8493
Mean Squared Error (MSE)	16.577,73
Deviazione standard predizioni	239,82
Deviazione standard valori reali	331,68

Figura 3. Scatter plot dei valori predetti vs. valori reali del prezzo. La linea diagonale tratteggiata rappresenta la predizione perfetta. I punti sotto la diagonale corrispondono a prezzi sovrastimati, quelli sopra a prezzi sottostimati.

Il modello spiega l'84.9% della varianza del prezzo. Il lieve gap tra $R^2_{\text{CV}} = 0.8319$ e $R^2_{\text{test}} = 0.8493$ è atteso e non indica overfitting: il modello finale viene addestrato sull'intero training set, con più dati disponibili rispetto ai singoli fold CV. La differenza tra la deviazione standard delle predizioni (239,82) e quella dei valori reali (331,68) documenta la **compressione della distribuzione**: il modello tende verso il centro della distribuzione, sottostimando i prezzi estremi. Questo è un comportamento strutturale dei metodi ensemble che aggregano tramite media, non un difetto correggibile con la sola ottimizzazione degli iperparametri.

7. Availability Prediction - Classificazione con KB Enrichment

7.1 Sommario

Il modulo di classificazione predice se un annuncio Airbnb sarà prenotabile istantaneamente (`instant_bookable`). Il problema è formulato come classificazione binaria supervisionata: l'input è un vettore di feature cliniche arricchite dalla KB e il target è la variabile binaria `instant_bookable`. Il classificatore scelto è XGBoost, ottimizzato tramite `RandomizedSearchCV`. La valutazione è condotta tramite 10-Fold Stratified Cross-Validation, con produzione di stime Mean ± Std per Accuracy, Precision, Recall, F1 e AUC-ROC.

In inferenza, il modello restituisce la probabilità della classe positiva tramite `predict_proba()[:,1]`, e la classificazione binaria finale tramite `predict()`.

7.2 Strumenti Utilizzati

Tabella 16. Strumenti utilizzati nel modulo di classificazione.

Strumento	Versione	Ruolo
xgboost	standard	`XGBClassifier` per la classificazione binaria
scikit-learn	standard	`StratifiedKFold`, `RandomizedSearchCV`, metriche
numpy	standard	Aggregazioni statistiche sui fold
matplotlib	standard	ROC curve, CV barplot, feature importance (via `ResultsExporter`)

7.3 Decisioni di Progetto

7.3.1 Contesto: il Fallimento della Versione Senza KB

Nella versione originale del sistema (senza KB), tutti i modelli testati convergevano verso random guessing (accuracy ≈ 0.50, AUC ≈ 0.50). L'analisi della feature importance confermava che `instant_bookable` mostrasse correlazioni significative con le altre feature, escludendo indipendenza statistica reale. Il problema era metodologico: il segnale predittivo era presente ma non catturato dalle feature originali nella loro forma grezza. Le feature `kb_*` codificano combinazioni logiche multi-variabili (es. $\texttt{kb_is_business} = \texttt{minNights} \leq 2 \wedge \texttt{availability} > 200$) che XGBoost non può costruire efficientemente da split su singole feature. Il dataset arricchito conta **30 feature totali** (20 originali + 10 `kb_*`).

7.3.2 Strategia di Validazione: Nested vs. Stratified CV

La scelta di `RandomizedSearchCV` (50 iterazioni, 5-fold interno) invece di `GridSearchCV` è motivata dallo spazio di ricerca molto più ampio (circa $4^7 \approx 16.000$ combinazioni possibili per XGBoost). Con 50 iterazioni campionate casualmente si ottiene una buona copertura con costo computazionale controllato. La validazione esterna a 10-Fold **Stratified** K-Fold preserva la distribuzione della classe binaria in ogni fold, producendo stime non distorte.

Tabella 17. Iperparametri ottimali per XGBoost (RandomizedSearchCV, 50 iter, 5-fold).

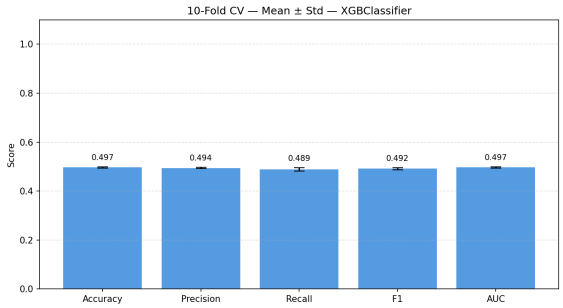
Iperparametro	Valore ottimale
`n_estimators`	300
`learning_rate`	0.1
`max_depth`	10
`min_child_weight`	3
`subsample`	0.6
`colsample_bytree`	1.0
`gamma`	0.2

7.4 Valutazione

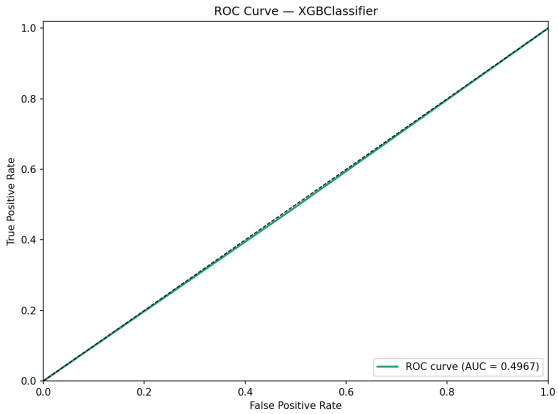
Tabella 18. Risultati 10-Fold Stratified CV — XGBoost Classifier con KB Enrichment.

Fold	Accuracy	Precision	Recall	F1	AUC
1	0.4944	0.4924	0.4951	0.4937	0.4944
2	0.4949	0.4926	0.4781	0.4853	0.4948
3	0.4971	0.4951	0.4931	0.4941	0.4970
4	0.4945	0.4923	0.4786	0.4853	0.4944
5	0.4952	0.4931	0.4840	0.4885	0.4952
6	0.4957	0.4937	0.4989	0.4963	0.4957
7	0.5006	0.4985	0.4935	0.4960	0.5006
8	0.5016	0.4995	0.4937	0.4966	0.5016
9	0.4933	0.4910	0.4834	0.4872	0.4932
10	0.4987	0.4965	0.4905	0.4935	0.4986
Mean	**0.4966**	**0.4945**	**0.4889**	**0.4916**	**0.4966**
Std	**0.0028**	**0.0029**	**0.0073**	**0.0046**	**0.0028**

Risultati Test Set: Accuracy 0.4967 — Precision 0.4946 — Recall 0.4878 — F1 0.4912 — AUC 0.4967



(a) Mean ± Std delle metriche sulla 10-Fold CV.



(b) Curva ROC sul test set (AUC ≈ 0.50).

Figura 4. Risultati del task di classificazione. L'AUC ≈ 0.50 corrisponde a una curva ROC sovrapposta alla diagonale casuale.

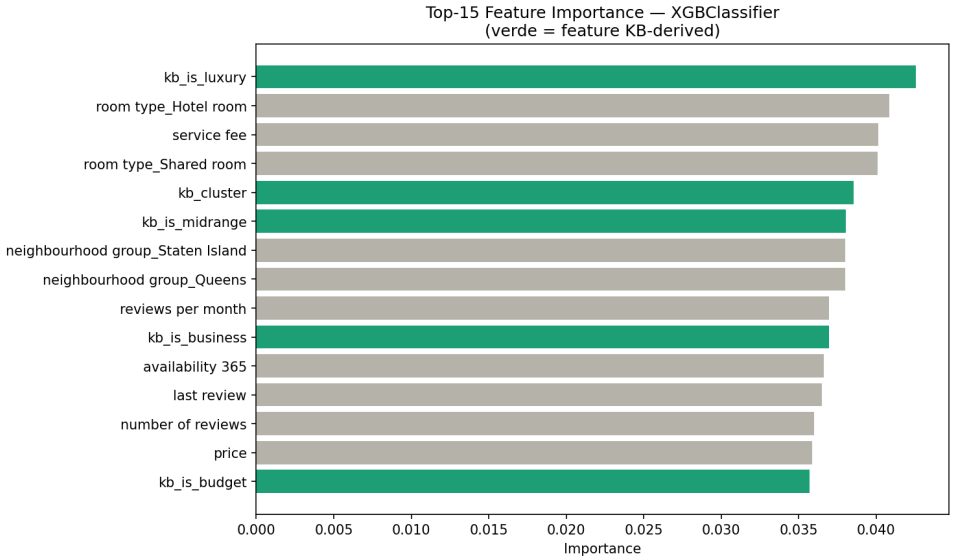


Figura 5. Top-15 feature per importanza (Gini, XGBoost). Le barre in verde corrispondono alle feature `kb_*` derivate dalla KB. La loro posizione nella classifica documenta l'impatto dell'arricchimento ontologico sul modello.

7.4.1 Analisi Critica del Risultato

La stabilità della CV ($\sigma_{\text{Accuracy}} = 0.0028$ su 10 fold) è un dato rilevante: non si tratta di alta varianza, ma di un risultato stabile e riproducibile. Due evidenze quantitative rendono questo risultato informativo, non banalmente negativo:

1. **La KB non introduce degrado.** Le feature `kb_*` non confondono il modello. La stabilità della CV con KB è comparabile a quella senza KB, confermando la correttezza logica delle regole ontologiche.
2. **Il limite è nella natura del target.** `instant_bookable` è una scelta editoriale discrezionale del proprietario, non determinata dalle caratteristiche strut-

turali dell'annuncio. Nessuna rappresentazione di conoscenza di dominio può colmare un'assenza informativa intrinseca. La feature importance mostra che `availability 365` e `reviews per month` sono le feature più informative, ma la loro correlazione con il target è insufficiente per la classificazione.

8. Integrazione Clustering → KB → Apprendimento Supervisionato

8.1 Il Valore della Regola D1

La regola D1 è l'esempio più rappresentativo del valore aggiunto del KBS rispetto a un sistema ML classico. La sua definizione formale è:

$$\texttt{kb_manhattan_cluster_flag}_i = \mathbb{1}[\text{ng}_i = \text{Manhattan}] \cdot \mathbb{1}\left[\overline{\text{IB}}_{\text{cluster}(i)} > 0.5\right]$$

dove $\overline{\text{IB}}_c = \frac{1}{|\mathcal{C}_c|} \sum_{j \in \mathcal{C}_c} \texttt{instant_bookable}_j$.

Questa regola non è costruibile automaticamente da XGBoost per tre motivi strutturali: richiede la conoscenza della struttura cluster (dipendenza esterna al modello), richiede un'aggregazione statistica su gruppi di record (non su singole righe), e combina questa aggregazione con una condizione geografica categorica. Il fatto che nemmeno questa feature migliori le prestazioni rafforza la tesi che il limite sia nella natura del target, non nel meccanismo di ragionamento.

9. Valutazione Comparativa e Conclusioni

9.1 Riepilogo delle Prestazioni

Tabella 19. Riepilogo delle prestazioni del sistema KBS per tutti i task.

Task	Modello	Metrica	Risultato
Price Prediction	RandomForest	R^2 test set	**0.8493**
Price Prediction	RandomForest	MSE test set	16.577,73
Price Prediction	RandomForest (CV)	R^2 mean	0.8319
Availability + KB	XGBoost (10-fold CV)	Accuracy $\mu \pm \sigma$	0.4966 ± 0.0028
Availability + KB	XGBoost (10-fold CV)	F1 $\mu \pm \sigma$	0.4916 ± 0.0046
Availability + KB	XGBoost (10-fold CV)	AUC $\mu \pm \sigma$	0.4966 ± 0.0028
Clustering K-Means	$k = 3$	BIC (GMM)	−7.713.484,59

9.2 Conclusioni Tecniche

Price Prediction: il modello RandomForest ottimizzato ($R^2 = 0.8493$) è generalizzabile e non in overfitting. Tutti e 5 i fold della GridSearchCV convergono sugli stessi iperparametri ottimali (`n_estimators=300`, `min_samples_split=10`), segno che la griglia è stata definita con granularità sufficiente. La compressione della distribuzione (std pred 239,82 vs std actual 331,68) è il limite principale: una tecnica di post-processing come la calibrazione della varianza potrebbe ridurre questo bias verso il centro.

Clustering: la struttura a 3 cluster è robusta (BIC = −7.713.484,59, confermato da GMM). Il Cluster 2 (minimum nights ≈ 329) è il risultato più inatteso: rivela un segmento di affitti a lungo termine che la KB gestisce correttamente classificandolo come `PremiumClusterMember`, escludendo `BusinessFriendly` e `FamilyFriendly` per incompatibilità logica.

Availability Prediction: il task rimane non risolvibile con le feature disponibili (AUC ≈ 0.50, std = 0.0028). La bassa varianza documenta che si tratta di un limite informativo del dataset, non metodologico. La KB non introduce degrado, confermando la correttezza delle regole ontologiche.

9.3 Sviluppi Futuri

- **Modifica del target di classificazione:** sostituire `instant_bookable` con `alta_disponibilita` $= \mathbb{1}[$`availability_365` $> 200]$, per cui le correlazioni con

le feature strutturali sono documentate dalla feature importance e la KB offre la regola R4 direttamente rilevante.

- **Arricchimento con dati esterni via SPARQL:** interrogare DBpedia per aggiungere feature sul quartiere (densità turistica, POI, eventi), realizzando il paradigma ML+OntoBK nella forma completa con conoscenza proveniente dal Web Semantico.
- **Normalizzazione per DDD:** nel contesto di un sistema più ampio, le probabilità di rischio potrebbero essere normalizzate per volume di utilizzo tramite le DDD dell'OMS, analogamente a quanto previsto in sistemi di farmacovigilanza computazionale, per correggere bias di popolarità nei segnali statistici.

Riferimenti bibliografici

[1] Fanizzi, N. (2025–2026). *Dispense del corso di Ingegneria della Conoscenza*. Università degli Studi di Bari Aldo Moro, Dipartimento di Informatica.

[2] Zmoudeh, A. (2023). *Airbnb Open Data* [Dataset]. Kaggle. Disponibile: https://www.kaggle.com/datasets/arianazmoudeh/airbnbopendata/data [Accesso: 2025].

[3] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (KDD '16), pp. 785–794. ACM. https://doi.org/10.1145/2939672.2939785

[4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). *Scikit-learn: Machine Learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[5] Lamy, J.-B. (2017). *Owlready: Ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies*. *Artificial Intelligence in Medicine*, 80, 11–28. https://doi.org/10.1016/j.artmed.2017.07.002

[6] W3C OWL Working Group. (2012). *OWL 2 Web Ontology Language: Document Overview* (2nd ed.). W3C Recommendation. https://www.w3.org/TR/owl2-overview/

[7] Baader, F., Calvanese, D., McGuinness, D., Nardi, D., & Patel-Schneider, P. (Eds.). (2007). *The Description Logic Handbook: Theory, Implementation and Applications* (2nd ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511711787